



modelimportance: Evaluating model importance within a multi-model ensemble in R

Minsu Kim 

University of Massachusetts Amherst

Li Shandross 

University of Massachusetts Amherst

Evan L. Ray 

University of Massachusetts Amherst

Nicholas G. Reich 

University of Massachusetts Amherst

Abstract

Ensemble forecasts are commonly used to support decision-making and policy planning across various fields because they often offer improved accuracy and stability compared to individual models. While a range of ensemble methods exist, understanding how much each constituent model contributes to the resulting ensemble performance is a separate but important consideration. The R package **modelimportance** addresses this need by providing a set of tools to quantify the importance of each component model within an ensemble. This functionality complements existing resources in the ‘**hubverse**’ framework — a collection of open-source software, tools and data standards developed to promote collaborative modeling hub efforts and simplify their setup and operation — by extending the capabilities of **hubEnsembles** for ensemble construction and **hubEvals** for ensemble evaluation. Our package supports multiple ensemble methods and multiple model importance metrics for both point and probabilistic forecasts. Additionally, the software offers customizable options for handling missing values. These features enable the package to serve as a versatile tool for researchers and practitioners. It helps not only in constructing an effective ensemble model across a wide range of forecasting tasks, but also in understanding the role of each model within the ensemble and gaining insights into individual models themselves. Following the ‘**hubverse**’ standards enables seamless integration and flexibility with other forecasting tools and systems, allowing many analyses to be performed on existing hubs.

Keywords: ensemble, forecast, prediction, model importance, Shapley value, R.

1. Introduction

Ensemble forecasting is a method to produce a single, consolidated prediction by combining forecasts generated from different models. While individual models' strengths and weaknesses are pronounced within ensembles, the combination of many models tend to offset each other and create an ensemble forecast that is more robust and accurate than any single component model (Gneiting and Raftery 2005; Hastie *et al.* 2001). Specifically, ensembles effectively mitigate the bias and variance arising from the predictions of individual models by averaging them out, and aggregating in this way can reduce prediction errors and improve overall performance. Enhanced prediction accuracy and robustness enable the achievement of more reliable predictions, thereby improving decision-making. For this reason, ensemble forecasting is widely used across various domains such as weather forecasting (Jordan A. Guerra *et al.* 2020; Gneiting and Raftery 2005), financial modeling (Sun *et al.* 2020; He *et al.* 2023), and infectious disease outbreak forecasting (Ray and Reich 2018; Reich *et al.* 2019; Lutz *et al.* 2019; Viboud *et al.* 2018) to name a few. For example, throughout the COVID-19 pandemic, the US COVID-19 Forecast Hub collected individual models developed by over 90 different research groups and built a probabilistic ensemble forecasting model for COVID-19 cases, hospitalizations, and deaths in the US, based on those models' predictions. The ensemble model served as the official short-term forecasts for the US Centers for Disease Control and Prevention (CDC) (Cramer *et al.* 2022).

A variety of ensemble construction methods have been proposed, ranging from simple averaging to more sophisticated approaches based on performance-weighted averaging, Bayesian model averaging, stacking, linear opinion pools, and machine learning (Wang *et al.* 2023; Reich *et al.* 2019; Ranjan and Gneiting 2010). In the 'hubverse' framework, a community-maintained ecosystem of open software and data standards for collaborative forecasting hubs (Consortium of Infectious Disease Modeling Hubs 2024; Kerr *et al.* 2025), the **hubEnsembles** (Shandross *et al.* (2026b)) provides a standardized approach to constructing ensembles via simple averaging and linear pooling, which the **modelimportance** builds upon rather than duplicating. Regardless of the ensemble construction method, a separate but important consideration is to understand how much each constituent model contributes to the resulting ensemble performance, and this lies at the core of our package. These insights will help to identify the most influential models, which can inform model selection, weighting, and refinement strategies (Figure 1).

The quality of forecasts is assessed by evaluating their errors, biases, sharpness, and/or calibration using different scoring metrics. The selection of scoring metrics depends on the type of forecast: point forecasts (e.g., mean, median) and probabilistic forecasts (e.g., quantiles, probability mass function). Commonly used assessment tools for point forecasts are the mean absolute error (MAE) and the mean squared error (MSE), which calculate the average magnitude of forecast errors. Scoring metrics for probabilistic forecasts consider the uncertainty and variability in predictions and provide concise evaluations through numerical scores (Gneiting and Raftery 2007). Some examples include the weighted interval score (WIS), the continuous ranked probability score (CRPS), and the log score (Bracher *et al.* 2021). (Note that CRPS is a general scoring rule that can be computed either analytically in closed form or numerically from samples, and WIS is a quantile-based approximation of CRPS.)

Several R packages have been developed to evaluate forecast quality across both point and probabilistic settings. To name a few, the **fable** package (O'Hara-Wild *et al.* 2024) is widely used for univariate time series forecasting and includes functions for accuracy measurement. The **Metrics** (Hamner and Frasco 2018) and **MLmetrics** (Yan 2024) provide a wide range

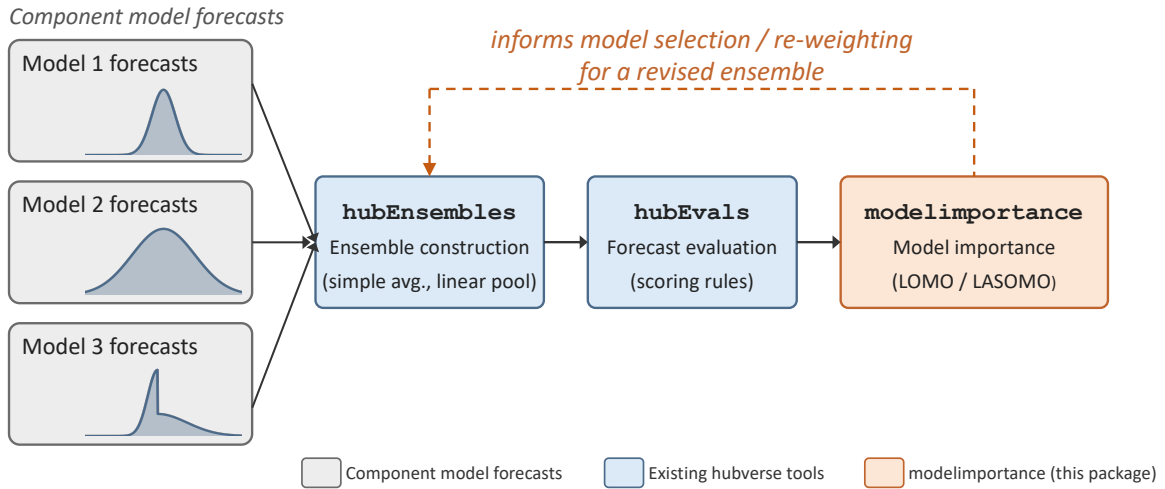


Figure 1: Example workflow illustrating how **modelimportance** fits within the hubverse ensemble forecasting pipeline. Component model forecasts are combined into an ensemble using **hubEnsembles**, and the resulting ensemble is scored using **hubEvals**. **modelimportance** then quantifies each component model's contribution to ensemble performance. These importance scores can, in turn, inform the selection or re-weighting of component models when constructing a revised ensemble (dashed feedback arrow), closing the loop between model evaluation and ensemble construction.

of performance metrics specifically designed for evaluating machine learning models. The **scoringRules** (Jordan *et al.* 2019) package offers a comprehensive set of proper scoring rules for evaluating probabilistic forecasts and supports both univariate and multivariate settings. The **scoringutils** (Bosse *et al.* 2024) package offers additional features to the functionality provided by **scoringRules**, which makes it more useful for certain tasks, such as summarizing, comparing, and visualizing forecast performance. These packages have been valuable to evaluate individual models as independent entities, using performance metrics selected for each specific situation or problem type. However, they do not measure the individual models' contributions to the enhanced predictive accuracy when used as part of an ensemble. Building on our prior methodological study (Kim *et al.* 2026), we emphasize that strong standalone model performance does not automatically imply a large positive contribution once the model is used within an ensemble. The **modelimportance** package operationalizes this idea in software, providing tools to evaluate the role of each model as an ensemble member within an ensemble model, rather than focusing on the individual predictive performance per se.

In ensemble forecasting, certain models contribute more significantly to the overall predictions than others. Assessing the impact of each component model on ensemble predictions is methodologically similar to determining variable importance in traditional regression and machine learning models, where variable importance measures evaluate how much individual variables improve the accuracy of the model's predictive performance or reduce the average loss. R packages such as **randomForest** (Liaw and Wiener 2002), **caret** (Kuhn 2008), **xgboost** (Chen *et al.* 2024), and **gbm** (Ridgeway and Developers 2024) implement these functions for different types of models: random forest models, general machine learning models, extreme gradient boosting models, and generalized boosted regression models, respectively. These packages

focus on feature-level importance within a single model and do not measure the contribution of individual models within an ensemble. In contrast, the tools in **modelimportance** quantify how each component model helps enhance the ensemble model’s predictive performance. They assign numerical scores to each model using a forecast accuracy metric selected based on the forecast type.

Our methods are based on the concept of Shapley values in cooperative game theory, which measure a player’s average contribution to the game’s overall outcome (Shapley (1953)). There are several approaches that utilize Shapley values in black-box machine learning models to understand how each feature affects the model’s predictive power (Lundberg and Lee (2017); Lundberg *et al.* (2020); Covert *et al.* (2020)). Related work by Lipiecki *et al.* (2024) used Shapley-based attribution to study contributions of component models in a combined ensemble after postprocessing point forecasts into probabilistic forecasts. That methodology was later implemented in **PostForecasts.jl** (Lipiecki and Weron 2025). **PostForecasts.jl** is conceptually aligned with our work in that both approaches leverage Shapley values to evaluate component forecasters. However, while **PostForecasts.jl** was developed for Julia users originally focusing on energy economics, our R package focuses on epidemiological forecasts and offers a broader range of capabilities, such as handling missing forecasts, compatibility with the hubverse forecasting ecosystem, and rich tools for summarizing and visualizing importance scores across tasks.

These capabilities are particularly useful for hub organizers, who oversee collaborative forecasting systems that combine submissions from many teams into a single ensemble forecast (Shandross *et al.* 2026a). Examples include efforts coordinated by the US CDC and the European Centre for Disease Prevention and Control. By quantifying each component model’s contribution, **modelimportance** can support evidence-based decisions about ensemble design and maintenance. The package follows conventions defined by the hubverse. Using these shared data structures allows our package to be seamlessly integrated with existing hub pipelines and deployed across multiple active hubs. We note that there are 31 hubs as of March 2026, including model development and training hubs.

We highlight some development practices we employed, such as unit testing of individual functions, object-oriented programming using S3 classes, continuous integration testing on different operating systems, and independent code review by peer developers. This emphasis on quality control is a key strength of this work.

The paper proceeds as follows. Section 2 describes (a) how the **modelimportance** package relates to the hubverse framework, including its dependencies, (b) the model output formats defined within hubverse, and (c) the structure of data presentation for both forecasts and actual observations. Section 3 presents two algorithms implemented in **modelimportance** for calculating the model importance metric: leave-one-model-out and leave-all-subsets-of-models-out. Section 4 demonstrates the various functionalities **modelimportance** supports. Section 5 describes the S3 class structure and related methods implemented in the package, followed by examples of their usage in Section 6. Section 7 discusses computational complexity and strategies for efficient computation. Section 8 highlights our quality assurance measures and the code’s availability, and then we close this paper with a summary and a discussion of possible extensions.

2. Data

2.1. Dependencies and related software

The **modelimportance** package is designed to work with the hubverse framework and, accordingly, depends on several packages in the hubverse ecosystem, such as **hubUtils** (Krystalli and Shandross (2025)), **hubEnsembles** (Shandross *et al.* (2026b)), and **hubEvals** (Reich *et al.* (2025)). **modelimportance** uses a `model_out_tbl` S3 class as the model output format defined in **hubUtils**, which consists of utility functions to standardize prediction files and data formats (details in Section 2.2). Ensembling predictions from multiple models relies on **hubEnsembles**, which offers a broadly applicable framework to construct multi-model ensembles using various ensemble methods. Calculation of forecast accuracy using various metrics is based on **hubEvals**, which internally leverages **scoringutils**.

We derived example datasets used for testing and demonstration purposes (see Section 6) from the **hubExamples** (Ray *et al.* (2025)) package, which provides example datasets in the hubverse format. The dataset were locally stored for reproducibility within the package distribution, instead of requiring the **hubExamples** package as a dependency.

For visualization of model importance scores, standard **ggplot2** (Wickham 2016) functions can be applied. Parallel computing is supported by the **furrr** (Vaughan and Dancho 2022) and **future** (Bengtsson 2021) packages, which allow users to specify the number of cores to use for parallel processing when calculating model importance scores across many tasks.

2.2. Model output format

Model outputs are structured in a tabular format designed specifically for predictions, which is a formal S3 object called `model_out_tbl`. In the hubverse standard, each row represents an individual prediction or a component of a prediction for a single task. More details about that prediction or prediction component are described in multiple columns through which one can identify the unique label assigned to each forecasting model, task characteristics, prediction representation type, and predicted values (Shandross *et al.* 2026a). To elaborate on the task characteristics, each prediction task corresponds to a specific forecasting problem and it can be described by a set of task ID variables. Examples of such variables include a date on which forecasts are generated, the target to predict (e.g., flu-related incident deaths, cases, or hospitalizations), and the prediction horizon, which is the length of time into the future from the point when a model generate its forecast, for a specific location on a certain target date. Table 1 illustrates short-term forecasts of weekly incident influenza hospitalizations in the US for Massachusetts, generated by the model ‘Flusight-baseline’ on December 17, 2022, in the `model_out_tbl` format. The `model_id` column lists a uniquely identified model name. The `reference_date`, `target`, `horizon`, `location`, and `target_end_date` columns are collectively referred to as the task ID variables, which together defines the task characteristics. Note that the forecast generation date and the target date for which the prediction is made are mapped to the `reference_date` and `target_end_date` columns, respectively, and the location is represented based on the FIPS code (e.g., ‘25’ for Massachusetts). The time length to the `target_end_date`, which is the number of weeks ahead from the `reference_date`, is

model_id	reference_date	target	horizon	location	target_end_date	output_type	output_type_id	value
Flusight-baseline	2022-12-17	wk inc flu hosp	1	25	2022-12-24	quantile	0.05	496
Flusight-baseline	2022-12-17	wk inc flu hosp	1	25	2022-12-24	quantile	0.1	536
Flusight-baseline	2022-12-17	wk inc flu hosp	1	25	2022-12-24	quantile	0.25	566
Flusight-baseline	2022-12-17	wk inc flu hosp	1	25	2022-12-24	quantile	0.5	582
Flusight-baseline	2022-12-17	wk inc flu hosp	1	25	2022-12-24	quantile	0.75	598
Flusight-baseline	2022-12-17	wk inc flu hosp	1	25	2022-12-24	quantile	0.9	629
Flusight-baseline	2022-12-17	wk inc flu hosp	1	25	2022-12-24	quantile	0.95	668
Flusight-baseline	2022-12-17	wk inc flu hosp	2	25	2022-12-31	quantile	0.05	454
Flusight-baseline	2022-12-17	wk inc flu hosp	2	25	2022-12-31	quantile	0.1	518
Flusight-baseline	2022-12-17	wk inc flu hosp	2	25	2022-12-31	quantile	0.25	558

Table 1: Example of the model output for incident influenza hospitalizations (top 10 rows) extracted from `forecast_data_example` bundled in the `modelimportance` package, which is originally from `hubExamples` package.

indicated in the `horizon` column. The prediction representation is specified as ‘quantile’ in the `output_type` column, and details are represented in the `output_type_id` column with seven quantiles of 0.05, 0.1, 0.25, 0.5, 0.75, 0.9, and 0.95 for each target end date. The predicted value corresponding to each quantile is recorded in the `value` column.

Figure 2 visualizes the information on the prediction task provided by Table 1 for three models. For each model, the quantile-based forecasts are shown for the target end dates of December 24, 2022 (horizon 1), December 31, 2022 (horizon 2), and January 07, 2023 (horizon 3), which were made on December 17, 2022 based on the historical data available as of that date. The prediction intervals defined by the lowest and highest quantiles (0.05 and 0.95) represent the uncertainty of the predictions. To give a brief interpretation, the Flusight-baseline model under-predicted the outcomes for the first two target dates (horizon 1 and 2), but it over-predicted the outcome for the last target date (horizon 3). Its prediction intervals are narrow compared to the other two models, which indicates that it is more confident about its predictions. However, two of three prediction intervals (horizons 1 and 2) failed to cover the eventually observed values, implying that the model was overconfident and/or biased.

2.3. Forecast data representation

Generally, quantitative forecasts can be categorized as being either point forecasts or probabilistic forecasts. For a specific prediction task, point forecasts, represented by a single predicted value, provide a clear and concise prediction, making them easy to interpret and communicate. Probabilistic forecasts, on the other hand, provide a probability distribution over possible future values, which inherently involves uncertainty. They are represented in various ways, such as probability mass functions (pmf), cumulative distribution functions (cdf), samples, or probability quantiles (or intervals). Note that there might be other ways, such as named distributions (e.g. `Normal(1,2)`) that are not currently supported by `hubverse`.

The `output_type` and `output_type_id` columns in the `hubverse` model output format specify the forecast structure. In the `modelimportance` package, model outputs may only contain one `output_type` of ‘mean’, ‘median’, ‘quantile’, or ‘pmf’: ‘mean’ or ‘median’ for point forecasts and ‘quantile’ or ‘pmf’ for probabilistic forecasts.

As aforementioned, `output_type_id` column identifies additional detailed information, such as specific quantile levels (e.g., “0.1”, “0.25”, “0.5”, “0.75”, and “0.9”) for the ‘quantile’

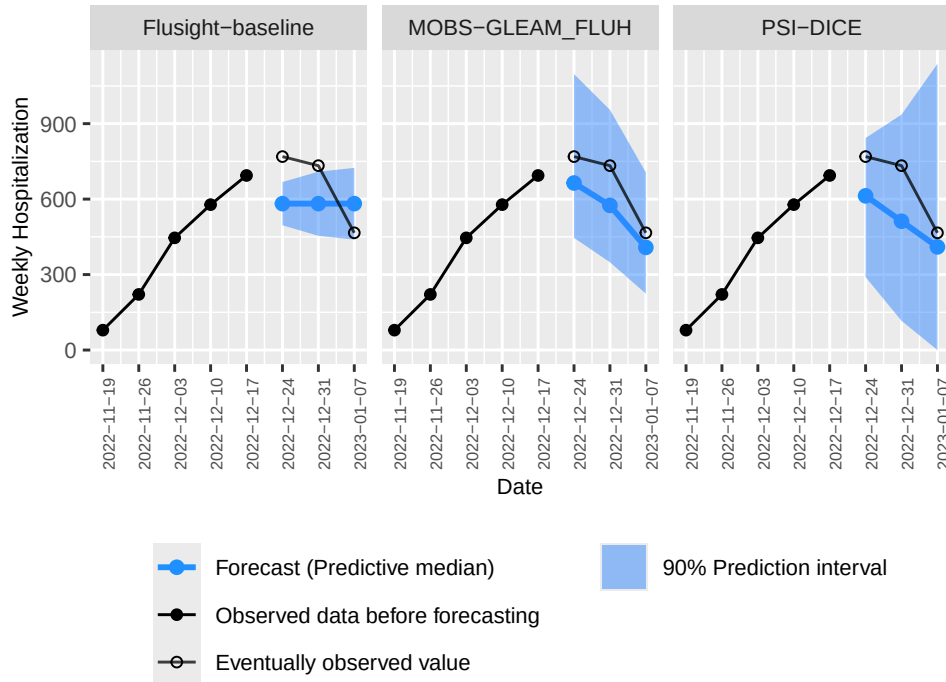


Figure 2: Example plot of three distributional forecasts corresponding to the model output for incident influenza hospitalizations shown in Table 1. Solid black dots indicate historically available data as of the forecast generation date, and open black circles indicate the eventually observed values. The blue dots represent predictive medians and the blue shaded area represents the corresponding 90% prediction interval defined by the 0.05 and 0.95 quantiles.

output type and categorical values (e.g., “low”, “moderate”, “high”, and “very high”) for the ‘pmf’ output type. The predicted values for **pmf** are constrained to be between 0 and 1, indicating the probability at each categorical level, while they are unbounded numeric otherwise. Different output types correspond to different scoring rules for evaluating a model’s prediction performance. Table 2 presents the output types and their associated scoring rules supported by the **modelimportance** package. We note that while the scoring rules are listed in the table using their conventional names, the package uses their negative values for evaluation (e.g., $-RSE$, $-WIS$) so that higher scores indicate better performance. This positively oriented scoring rule facilitates the interpretation of importance scores: positive values indicate that the model’s inclusion in the ensemble improves ensemble performance, whereas negative values indicate that it worsens it. Further, using positive values to indicate improved ensemble performance aligns with the convention of Shapely values.

2.4. Oracle output data

The **oracle_output_data** is a data frame that contains the ground truth values for the variables used to define modeling targets (Consortium of Infectious Disease Modeling Hubs 2024; Kerr *et al.* 2025). Its name originates from the concept of an oracle making a perfect prediction formatted similarly to model output. This type of data must follow the oracle output format defined in the hubverse standard, which includes independent task ID columns

Output Type	Scoring Rule	Description
mean	SE	Squared error (SE): the squared difference between the predicted value and the observed value
median	AE	Absolute error (AE): the absolute difference between the predicted value and the observed value
quantile	WIS	Weighted interval score (WIS): a quantile-based approximation of the continuous ranked probability score (CRPS) for evaluating quantile forecasts
pmf	Log Score	Logarithm of the probability assigned to the true outcome (LogScore)

Table 2: Pairs of output types and their associated scoring rules for evaluating prediction performance.

(e.g., `location`, `target_date`), the `output_type` column specifying the output type of the predictions and an `oracle_value` column for the observed values. As in the forecast data, if the `output_type` is either "quantile" or "pmf", the `output_type_id` column is often required to provide further identifying information.

The `model_out_tbl` and `oracle_output_data` must have the same task ID columns and `output_type`, including `output_type_id` if necessary, as the `oracle_output_data` is joined with the `model_out_tbl` based on these fields in order to score the forecast performance.

3. Method description and algorithms

This section provides a brief description of the leave one model out (LOMO) and leave all subsets of models out (LASOMO) algorithms, which are used to compute the model importance score. The basic idea of measuring the importance of each component model is to evaluate the change in ensemble performance when that model is included or excluded in the ensemble construction. More specifically, we compare the performance of an ensemble with and without a specific model for a specific task, and consider the difference in performance as the importance of that model for that task (Figure 3). We apply this idea across many tasks and then average the task-level values to obtain model-level summaries. The full theoretical development and additional experiments are provided in Kim *et al.* (2026).

LOMO involves creating an ensemble by excluding one component model from the entire set of models. Let $\mathcal{A}$ be a set of n models and F^i be a forecast produced by model i , where $i = 1, 2, \dots, n$. Each ensemble excludes exactly one model while including all the others. $F^{\mathcal{A}^{-i}}$ denotes the ensemble forecast constructed using all forecasts $F^{\mathcal{A}}$ except F^i . Model i 's importance score using LOMO is calculated as the difference in accuracy, as measured by a specific scoring rule, between $F^{\mathcal{A}^{-i}}$ and $F^{\mathcal{A}}$ (Algorithm 1). For example, when evaluating model 1 within an ensemble of three models ($n = 3$), LOMO creates an ensemble forecast $F^{\{2,3\}}$ using only F^2 and F^3 . The performance of this reduced ensemble is then compared to the full ensemble forecast $F^{\{1,2,3\}}$, which incorporates all three models. We note that a model can make an ensemble better or worse, and thus the importance score for model 1 can be positive or negative accordingly.

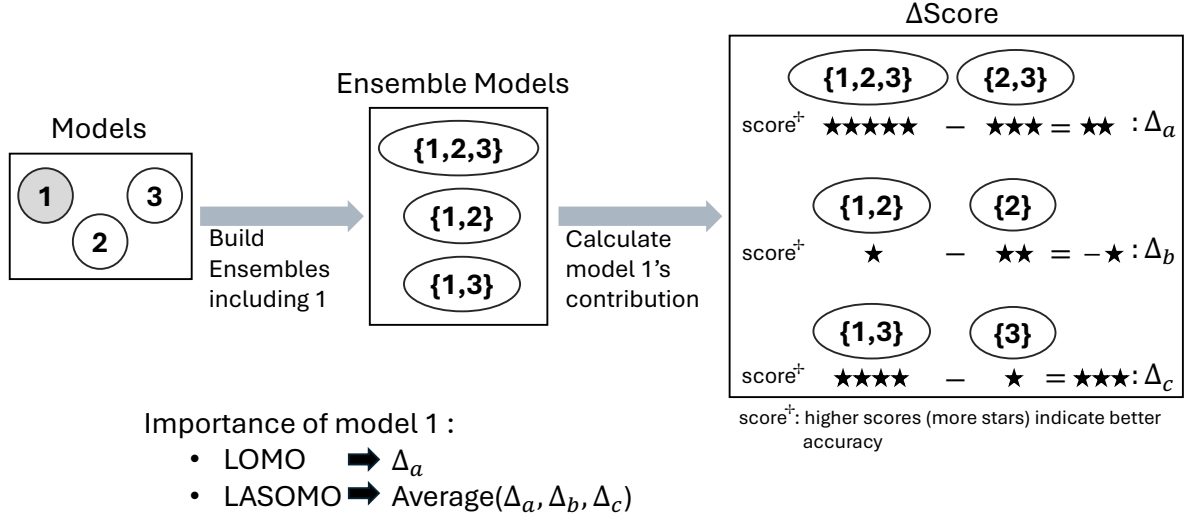


Figure 3: Conceptual illustration of measuring model importance in a three-model setting. Each circle represents a forecasting model with component model(s) shown inside. Stars indicate the performance score measured by a positively oriented scoring rule (e.g., $-WIS$), with more stars indicating better accuracy. LOMO computes a single performance difference by removing the target model 1 from the full ensemble. LASOMO leverages multiple performance differences across all subsets containing the model 1 and aggregates them.

On the other hand, LASOMO involves ensemble constructions from all possible subsets of models. For each subset S that does not contain the model i , $S \cup \{i\}$ plays the role of $\mathcal{A}$ in

Algorithm 1 Importance score calculation for one prediction task using leave one model out (LOMO) algorithm

Input:

- Set of n individual models, $\mathcal{A} = \{1, 2, \dots, n\}$, and their forecasts $\{F^1, F^2, \dots, F^n\}$
- Truth value, y
- Function g to build an ensemble

Output: Importance scores of all models, $\phi^{i, \text{lo mo}} (i = 1, 2, \dots, n)$

- 1: Assign a positively oriented scoring rule ψ to evaluate forecast skills, depending on the output type (Table 2)
 - 2: Create an ensemble forecast $F^{\mathcal{A}}$ using g : $F^{\mathcal{A}} \leftarrow g(\{F^1, F^2, \dots, F^n\})$
 - 3: Evaluate $F^{\mathcal{A}}$ using ψ : $\psi(F^{\mathcal{A}}, y)$.
 - 4: **for** each $i, (i = 1, 2, \dots, n)$ **do**
 - 5: $F^{\mathcal{A}^{-i}} \leftarrow g(\{F^j | j \neq i, j \in \mathcal{A}\})$
 - 6: Compute $\psi(F^{\mathcal{A}^{-i}}, y)$.
 - 7: $\phi^{i, \text{lo mo}} \leftarrow \psi(F^{\mathcal{A}}, y) - \psi(F^{\mathcal{A}^{-i}}, y)$
 - 8: **end for**
-

Algorithm 2 Importance score calculation for one prediction task using leave all subsets of models out (LASOMO) algorithm

Input:

- Set of n individual models, $\mathcal{A} = \{1, 2, \dots, n\}$, and their forecasts $\{F^1, F^2, \dots, F^n\}$
- Truth value, y
- Function g to build an ensemble
- Subset weighting option, either ‘equal’ or ‘perm_based’

Output: Importance scores of all models, $\phi^{i, \text{lasomo}}(i = 1, 2, \dots, n)$

- 1: Assign a positively oriented scoring rule ψ to evaluate forecast skills, depending on the output type (Table 2)
 - 2: **for** $i = 1$ to n **do**
 - 3: $\phi^{i, \text{lasomo}} \leftarrow 0$
 - 4: Make a list of non-empty subsets of $\mathcal{A}$ that does not contain i : $S_1, S_2, \dots, S_{2^{n-1}-1}$
 - 5: **for** $j = 1$ to $2^{n-1} - 1$ **do**
 - 6: Assign a weight to the subset S_j :
 - 7: **if** ‘equal’ subset weighting is selected **then**
 - 8: $\gamma_{S_j} \leftarrow \frac{1}{2^{n-1} - 1}$
 - 9: **else** (‘perm_based’ subset weighting)
 - 10: $\gamma_{S_j} \leftarrow \frac{1}{(n-1)\binom{n-1}{|S_j|}}$
 - 11: **end if**
 - 12: $F^{S_j} \leftarrow g(\{F^j | j \in S_j\})$
 - 13: $F^{S_j \cup \{i\}} \leftarrow g(\{F^i, F^j | j \in S_j\})$
 - 14: Compute $\psi(F^{S_j}, y)$ and $\psi(F^{S_j \cup \{i\}}, y)$
 - 15: $\phi^{i, \text{lasomo}} \leftarrow \phi^{i, \text{lasomo}} + \gamma_{S_j} \times [\psi(F^{S_j \cup \{i\}}, y) - \psi(F^{S_j}, y)]$
 - 16: **end for**
 - 17: **end for**
-

the LOMO; the score associated with the subset S is the difference of measures between F^S and $F^{S \cup \{i\}}$. Then, all scores are aggregated across all possible subsets that the model i does not belong to (Algorithm 2). For example, using the earlier setup of three forecast models, LASOMO considers three subsets, which we denote by $S_1 = \{2\}$, $S_2 = \{3\}$, and $S_3 = \{2, 3\}$, to calculate the importance score of model 1 (excluding all subsets that include model 1). The ensemble forecasts $F^{\{2\}}$, $F^{\{3\}}$, and $F^{\{2,3\}}$ are then compared to $F^{\{1,2\}}$, $F^{\{1,3\}}$, and $F^{\{1,2,3\}}$, respectively. The performance differences attributable to model 1’s inclusion are aggregated, which results in the importance score for model 1. We note that the subsets (e.g., S_1 , S_2 , and S_3) may have different weights during the aggregation process.

The **modelimportance** package offers two weighting options for subsets: one assigns equal (uniform) weights to all subsets, and the other assigns weights based on their size, similar to the definition of Shapley values, where a player’s average contribution is aggregated over all possible coalitions (or, equivalently, over all permutations of players) (Shapley (1953)). Users

can choose one to evaluate the contribution of each model in a manner suited to their preferred framework. Uniform weighting may be preferred when each subset size is of equal analytical interest, regardless of how many such subsets there are, while size-based weighting may be preferred when users want to maintain the original Shapley value interpretation, preventing numerous medium-sized subsets from dominating the importance scores. A detailed discussion of the differences between the two weighting schemes follows in the next section.

3.1. Comparison of weighting schemes in LASOMO

The differences in how the two weighting schemes influence the importance scores become more pronounced as the number of models increases. As described in [Algorithm 2](#), the formulas for their subset weights are

$$\frac{1}{2^{n-1} - 1} \quad \text{and} \quad \frac{1}{(n-1)\binom{n-1}{k}},$$

where k is the size of each subset and n is the total number of models. The equal scheme (left formula) treats all subsets equally, so medium-sized subsets have considerable influence in the final result, as there are many such subsets. In contrast, the permutation-based scheme (right formula) adjusts the weights according to the subset size, giving the greatest weight to both the smallest and largest subsets while assigning small weights to the mid-sized subsets. Moreover, the weights assigned to the mid-sized subsets under the permutation-based approach decrease much faster with n than those under the equal weighting scheme (see [Appendix A](#) for details). Consequently, when n is large, middle-sized subsets play a dominant role in determining the importance scores under the equal weighting scheme, whereas extreme-sized subsets primarily drive the scores under the permutation-based weighting approach.

Overall, the difference between the two weighting schemes is likely to arise mainly from the the extreme-sized subsets when n is large. This is because the weights given to the mid-sized subsets become increasingly similar, which are very small values on the order of 10^{-3} even when $n = 8$, while the weights assigned to the smallest and largest subsets remain substantially different ([Figure 4](#)).

4. Evaluating models with the `model_importance()` function

In this section, we describe the usage of the function `model_importance()`, where multiple options are available to customize the evaluation framework ([Table 3](#)).

The `model_importance()` function calculates the importance scores of ensemble component models based on their contributions to improving ensemble prediction accuracy for each prediction task. The function requires a minimum of two models per task, as with only one model, the importance score is not defined. The output of the function is a single data frame of importance scores combined across all tasks. If a model missed predictions for a specific task, an NA value will be assigned for that task.

```
> model_importance(
  forecast_data, oracle_output_data, ensemble_fun,
  importance_algorithm, subset_wt, min_log_score, ...
)
```

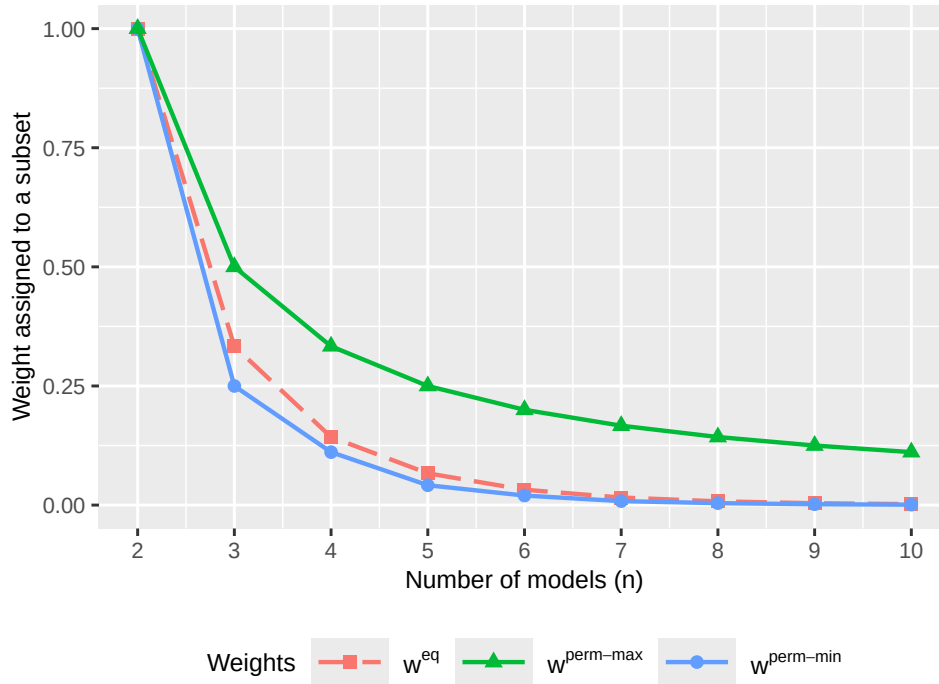


Figure 4: Comparison of weights assigned to a subset. The plot shows the weights assigned to a subset as the number of models n increases from 2 to 10. The red dashed line represents the weights under the equal weighting scheme, while the blue and green lines represent the minimum and maximum weights, respectively, under the permutation-based weighting scheme. The minimum weight occurs when the subset size is around $(n-1)/2$, and the maximum weight occurs when the subset size is $n-1$. As the number of models increases, the weights assigned by the two schemes become increasingly similar for mid-sized subsets whereas substantial differences remain for extreme-sized subsets.

The `forecast_data` parameter is a data frame of predictions that is automatically coerced to a `model_out_tbl` format (if it is not one already), which is the standard S3 class model output format defined by the hubverse convention. If coercion fails, users may need to manually convert their data using the `as_model_out_tbl()` function from **hubUtils**.

The `oracle_output_data` is a data frame containing the actual observed values of the variables used to specify modeling targets. Further details on formatting and purpose are provided in [Section 2.4](#).

The `ensemble_fun` argument specifies the ensemble method to use when evaluating model importance, which relies on implementations in the **hubEnsembles** package (Shandross *et al.* (2026b)). The currently supported methods are "simple_ensemble" and "linear_pool". The "simple_ensemble" method returns the average of the predicted values from all component models per prediction task defined by task IDs, `output_type`, and `output_type_id` columns. The default aggregation function for this method is "mean", but it can be customized by specifying additional arguments through `...`, such as `agg_fun="median"`. When "linear_pool" is selected, the ensemble is a linear opinion pool of its component models. This method supports only an `output_type` of "mean", "quantile", or "pmf".

Argument	Description	Possible Values	Default
<code>forecast_data</code>	Forecasts	Table of model output	N/A
<code>oracle_output_data</code>	Ground truth data	Table of oracle output	N/A
<code>ensemble_fun</code>	Ensemble method	"simple_ensemble", "linear_pool"	"simple_ensemble"
<code>importance_algorithm</code>	Algorithm to calculate importance	"lomo", "lasomo"	"lomo"
<code>subset_wt</code>	Method for assigning weight to subsets when using LASOMO algorithm	"equal", "perm_based"	"equal"
<code>min_log_score</code>	Minimum value to replace for log score	Non-positive numeric	-10
...	Optional arguments for "simple_ensemble"	Varies	<code>agg_fun="mean"</code>

Table 3: Description of the arguments for the `model_importance()` function, including their purpose, possible values, and default settings.

The `importance_algorithm` argument specifies the algorithm for model importance calculation, which can be either "lomo" (leave one model out) or "lasomo" (leave all subsets of models out). The `subset_wt` argument is employed only for the "lasomo" algorithm. This argument has two options: "equal" assigns equal weight to all subsets and "perm_based" assigns weight averaged over all possible permutations, as in the formula of Shapley values (Algorithm 2). The default values of `importance_algorithm` and `subset_wt` are "lomo" and "equal", respectively.

The `min_log_score` argument is relevant only for the `output_type` of "pmf", which uses Log Score as a scoring rule. It sets a minimum threshold for log scores to avoid issues with extremely low probabilities assigned to the true outcome, which can lead to undefined or negative infinite log scores. Any probability lower than this threshold will be adjusted to this minimum value before calculating the importance metric based on the log score. The default value is set to -10, following the CDC FluSight thresholding convention (Brooks *et al.* 2018; Reich *et al.* 2019). Users may choose a different value based on their practical needs.

The `model_importance()` function returns an object of S3 class `model_imp_tbl` of model important scores, with columns `model_id`, `reference_date`, `output_type`, and `importance`, along with any task ID columns (e.g., `location`, `horizon`, and `target_end_date`) present in the input `forecast_data`. Regardless of the original column name for the forecast generation date in the `forecast_data`, it is standardized to `reference_date` in the output. This standardization enables consistent handling of the forecast generation date across datasets, simplifying downstream processing. The `importance` column contains the calculated importance scores for each model and specific task, which are derived from the specified algorithm and ensemble method.

5. S3 class and methods

We have defined a custom S3 class, `model_imp_tbl` that represents the output of the `model_importance()` function, which extends the base `data.frame` class. Objects of this class enable dispatch for class-specific methods for printing, summarizing, and aggregating them across tasks.

5.1. Print method

`print()` provides all computation results in a clear and organized manner. It displays `model_id` and `importance` by grouped task IDs (e.g., `reference_date`, `location`, and `target_end_date`) to facilitate easy interpretation of the importance scores for each model across different tasks.

5.2. Summary method

`summary()` provides a concise summary of the importance scores, including key statistics such as the number of models and tasks evaluated. It also displays top-scoring models for a subset of tasks. Additional summary details are available by specifying individual elements of the summary object as follows:

- `.$all_tasks` lists all the unique tasks evaluated, which are defined by the combinations of task ID columns present in the input `forecast_data`. This allows users to identify the scope of the evaluation.
- `.$model_summary` provides each model's performance summary across tasks (e.g., the number of tasks each model submitted its forecast for, and the range of scores it achieved). This helps users understand the consistency and variability of each model's importance across different tasks.
- `.$task_winners` identifies which model is the best for each task based on the highest importance score across the full set of tasks. From this information, users can quickly identify the most important model in each task.

5.3. Aggregate method

`aggregate()` allows users to obtain an overall importance score for each model by aggregating its importance scores across all evaluated tasks. This `model_imp_tbl` class-specific method provides arguments that users can specify to customize the aggregation process, including the handling of NA values and the choice of summary function (Table 4).

The `by` argument specifies the grouping variable(s) for summarization. Its default is `"model_id"`, in which the `aggregate()` method summarizes importance scores for each model. Valid values for `by` include any combination of columns present in the `importance_scores` data frame.

The `na_action` argument allows for specifying how to handle NA values generated during importance score calculation for each task; these values occur when a model did not contribute to the ensemble prediction for a given task by missing its forecast submission. Three options

Argument	Description	Possible Values	Default
importance_scores	Model importance scores produced by model_importance()	data frame	N/A
by	Grouping variable(s) for summarization	grouping variable(s)	"model_id"
na_action	Method to handle NA values	"drop", "worst", "average"	"drop"
fun	Function to summarize importance scores	summary function	mean
...	Optional arguments for "fun"	depends on fun	N/A

Table 4: Description of the arguments for the **aggregate()** function for **model_imp_tbl** objects, including their purpose, possible values, and default settings.

are available: "worst", "average", and "drop". In each specific prediction task, if a model has any missing predictions, the "worst" option replaces the NA values with the smallest value among other models' importance metrics, while the "average" option replaces them with the average of the other models' importance metrics in that task. The "drop" option removes the NA values, which results in the exclusion of the model from the evaluation for that task.

The **fun** argument specifies a function used to summarize importance scores. An arithmetic mean (**fun = mean**) is the default, but other summary functions (e.g., **fun = median**) or user-defined functions may also be used. Additional arguments for the summary function **fun** can also be passed via **...** if needed (e.g., **fun = quantile**, **probs = 0.25** for a quartile summary).

The output returned by **aggregate()** is a data frame with columns **model_id** and **importance_score_<fun>**, where **<fun>** is the name of the summary function used (e.g., **importance_score_mean** when **fun = mean**). The output is sorted in descending order of the summarized importance scores, **importance_score_<fun>**.

6. Examples

In this section, we illustrate how to use the **model_importance()** function to evaluate the importance of component models within an ensemble. The examples show various combinations of the arguments described in [Section 4](#). Example forecast and target data are originally sourced from the **hubExamples** package, which provides sample datasets for multiple modeling hubs in the hubverse format.

6.1. Example data

Our example forecast data contains short-term predictions of weekly incident influenza hospitalizations in the US for Massachusetts (FIPS code 25) and Texas (FIPS code 48), generated on November 19, 2022. These forecasts are made for two target end dates, November 26,

2022 (horizon 1), and December 10, 2022 (horizon 3), and were produced by three models: ‘Flusight-baseline’, ‘MOBS-GLEAM_FLUH’, and ‘PSI-DICE’. The output type is `median` and the `output_type_id` column has NAs as no further specification is required for this output type. We have modified the example data slightly by removing some forecasts to demonstrate the handling of missing values. Therefore, MOBS-GLEAM_FLUH’s forecast for Massachusetts on November 26, 2022, and PSI-DICE’s forecast for Texas on December 10, 2022, are missing. We emphasize that this modification is made randomly and artificially to illustrate the impact of different handling approaches for missing forecasts and is not intended to provide a formal evaluation of model quality or performance.

```
> forecast_data |>
  knitr::kable(format = "latex", booktabs = TRUE) |>
  kable_styling(latex_options = c("scale_down", "hold_position"))
```

model_id	reference_date	target	horizon	location	target_end_date	output_type	output_type_id	value
Flusight-baseline	2022-11-19	wk inc flu hosp	1	25	2022-11-26	median	NA	51
Flusight-baseline	2022-11-19	wk inc flu hosp	3	25	2022-12-10	median	NA	51
Flusight-baseline	2022-11-19	wk inc flu hosp	1	48	2022-11-26	median	NA	1052
Flusight-baseline	2022-11-19	wk inc flu hosp	3	48	2022-12-10	median	NA	1052
MOBS-GLEAM_FLUH	2022-11-19	wk inc flu hosp	3	25	2022-12-10	median	NA	43
MOBS-GLEAM_FLUH	2022-11-19	wk inc flu hosp	1	48	2022-11-26	median	NA	1072
MOBS-GLEAM_FLUH	2022-11-19	wk inc flu hosp	3	48	2022-12-10	median	NA	688
PSI-DICE	2022-11-19	wk inc flu hosp	1	25	2022-11-26	median	NA	90
PSI-DICE	2022-11-19	wk inc flu hosp	3	25	2022-12-10	median	NA	159
PSI-DICE	2022-11-19	wk inc flu hosp	1	48	2022-11-26	median	NA	1226

The `forecast_data` output was rendered using the **knitr** (Xie 2025) package and styled with **kableExtra** (Zhu 2024) to improve readability, including automatic scaling to fit within page width.

The corresponding target data contains the observed hospitalization counts for these dates and locations.

```
> target_data

# A tibble: 4 x 4
  target_end_date target          location oracle_value
  <date>          <chr>          <chr>          <dbl>
1 2022-11-26      wk inc flu hosp 25             221
2 2022-11-26      wk inc flu hosp 48            1929
3 2022-12-10      wk inc flu hosp 25             578
4 2022-12-10      wk inc flu hosp 48            1781
```

When comparing the ground truth data and model predictions, we can see that forecasts for December 10, 2022 show larger deviations from the observed values compared to those for November 26, 2022. Thus, as we expect, prediction errors increase at longer horizons due to greater uncertainty. Additionally, the forecasts for Massachusetts are relatively more accurate compared to those for Texas, which tend to have higher errors (Figure 5).

6.2. Evaluation using LOMO algorithm

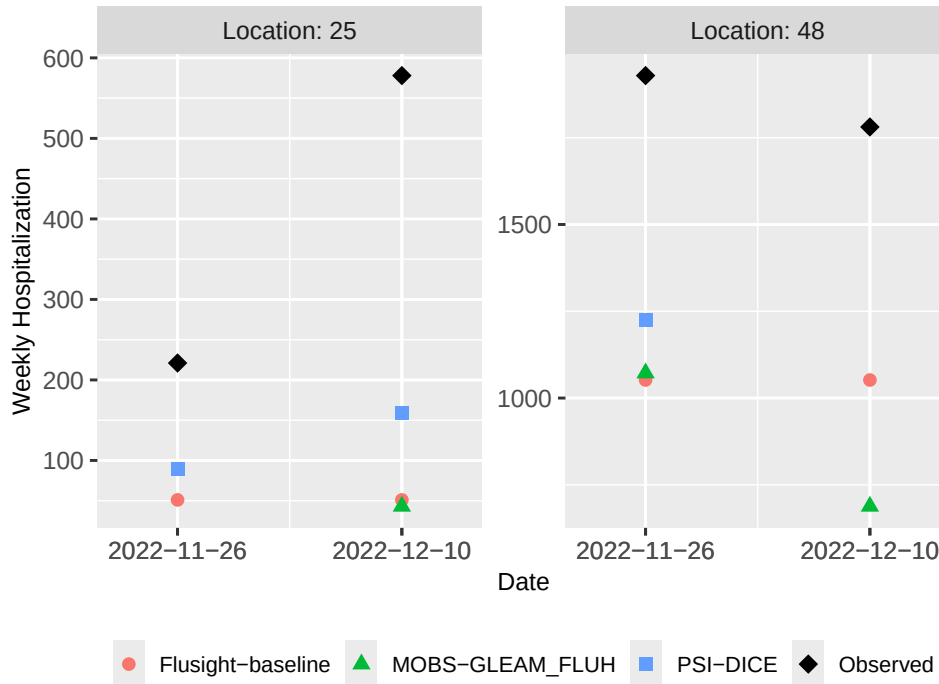


Figure 5: Plot of three point forecasts (median) and the eventually observed values from the `forecast_data` and `target_data` for weekly incident influenza hospitalizations in Massachusetts (FIPS code 25) and Texas (FIPS code 48). Colored dots indicate the forecasts by three models, generated on November 19, 2022. Open black circles indicate the eventually observed values. MOBS-GLEAM_FLUH's forecast for Massachusetts on November 26, 2022, and PSI-DICE's forecast for Texas on December 10, 2022, are not shown. These forecasts were manually excluded from the original example dataset for demonstration purposes.

We quantify the contribution of each model within the ensemble using the `model_importance()` function. The following code evaluates the importance of each ensemble member in the simple mean ensemble using the LOMO algorithm.

```
> scores_lomo <- model_importance(
  forecast_data = forecast_data,
  oracle_output_data = target_data,
  ensemble_fun = "simple_ensemble",
  importance_algorithm = "lomo"
)
```

This call also generates informative messages that summarize the input data, including the number of dates on which forecasts were produced, the number of models and their ids, and whether the prediction tasks meet the minimum model requirement, alongside a note on how to use parallel processing to speed up the computation when there are many models and tasks. A print out of these messages is shown below.

Evaluating forecasts from 2022-11-19 to 2022-11-19 (a total of 1 forecast date(s)).

The available model IDs are:

```

    Flusight-baseline
    MOBS-GLEAM_FLUH
    PSI-DICE
(a total of 3 models)

```

Note: This function uses 'furrr' and 'future' for parallelization.
To enable parallel execution, please set `future::plan(multisession)`.

All tasks meet the minimum model requirement of 2 models.

The function output is a data frame containing model ids and their corresponding importance scores for each prediction task, along with task id columns. (Note that we rounded the importance scores and renamed the columns for better readability in the output below, but the original column names are retained in the actual output data frame.)

```

> print(
  scores_lomo |>
    mutate(importance = round(importance, 2)) |>
    rename(ref_date = reference_date, h=horizon,
           loc=location, t_end_date=target_end_date,
           o_type=output_type, imp=importance)
)

```

Model importance result by task

```

-----
      model_id  ref_date      target h loc t_end_date o_type      imp
1 Flusight-baseline 2022-11-19 wk inc flu hosp 1 25 2022-11-26 median -19.50
2   MOBS-GLEAM_FLUH 2022-11-19 wk inc flu hosp 1 25 2022-11-26 median      NA
3      PSI-DICE 2022-11-19 wk inc flu hosp 1 25 2022-11-26 median   19.50
4 Flusight-baseline 2022-11-19 wk inc flu hosp 1 48 2022-11-26 median -32.33
5   MOBS-GLEAM_FLUH 2022-11-19 wk inc flu hosp 1 48 2022-11-26 median -22.33
6      PSI-DICE 2022-11-19 wk inc flu hosp 1 48 2022-11-26 median   54.67
7 Flusight-baseline 2022-11-19 wk inc flu hosp 3 25 2022-12-10 median -16.67
8   MOBS-GLEAM_FLUH 2022-11-19 wk inc flu hosp 3 25 2022-12-10 median -20.67
9      PSI-DICE 2022-11-19 wk inc flu hosp 3 25 2022-12-10 median   37.33
10 Flusight-baseline 2022-11-19 wk inc flu hosp 3 48 2022-12-10 median  182.00
11   MOBS-GLEAM_FLUH 2022-11-19 wk inc flu hosp 3 48 2022-12-10 median -182.00
12      PSI-DICE 2022-11-19 wk inc flu hosp 3 48 2022-12-10 median      NA

```

For models that missed forecasts for certain tasks, NA values were assigned in the importance column for those tasks.

Calling `summary()` shows that three models were used and four tasks were evaluated, along with a preview of the top-performing model for each task.

```

> summary(scores_lomo)

=== Summary of importance scores by task ===
Number of models: 3
Number of tasks: 4

=== Top scoring model by task for a subset of tasks =====
target horizon location target_end_date top_model importance

```

```

wk inc flu hosp 1 25 2022-11-26 PSI-DICE 19.50
wk inc flu hosp 1 48 2022-11-26 PSI-DICE 54.67
wk inc flu hosp 3 25 2022-12-10 PSI-DICE 37.33

```

```

-----
* More details are available in the summary object (e.g., $all_tasks, $model_summary,
  $task_winners).

```

As indicated in the output, more details about the summary are available through the summary object's elements as follows.

```

> s <- summary(scores_lomo)
> s$all_tasks

      target horizon location target_end_date
1 wk inc flu hosp      1      25      2022-11-26
2 wk inc flu hosp      1      48      2022-11-26
3 wk inc flu hosp      3      25      2022-12-10
4 wk inc flu hosp      3      48      2022-12-10

```

Each row represents a unique combination of task IDs, from which we verify that four different tasks were evaluated.

```

> s$model_summary

      model_id n_tasks min_importance max_importance n_NA
1 Flusight-baseline      4      -32.33      182.00      0
2 MOBS-GLEAM_FLUH      4     -182.00     -20.67      1
3 PSI-DICE      4      19.50      54.67      1

```

We observe that 'Flusight-baseline' submitted forecasts for all four tasks ($n_NA = 0$), while 'MOBS-GLEAM_FLUH' and 'PSI-DICE' submitted forecasts for only three tasks due to one missing forecast ($n_NA = 1$). Each model's importance scores vary across tasks. 'Flusight-baseline' shows the largest range of scores that includes a negative minimum value and a positive maximum value, while 'MOBS-GLEAM_FLUH' and 'PSI-DICE' have scores that are all positive or all negative across the three tasks they submitted forecasts for.

```

> s$task_winners

      target horizon location target_end_date      top_model max_score
1 wk inc flu hosp      1      25      2022-11-26      PSI-DICE      19.50
2 wk inc flu hosp      1      48      2022-11-26      PSI-DICE      54.67
3 wk inc flu hosp      3      25      2022-12-10      PSI-DICE      37.33
4 wk inc flu hosp      3      48      2022-12-10 Flusight-baseline      182.00

```

Models with the highest importance scores for each task are identified in the `top_model` column with their importance score in the `max_score` column. 'PSI-DICE' is the best model for three out of the four tasks, while 'Flusight-baseline' is the best for the remaining task.

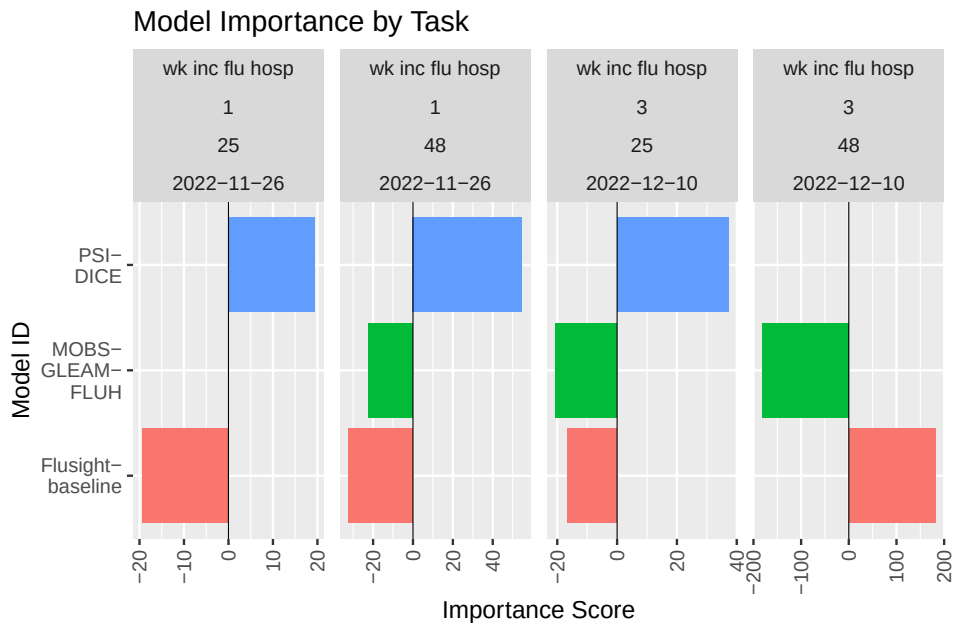
Visualization can be performed using functions from **ggplot2**. The following example shows a bar plot of importance scores across models and tasks, with panels faceted by combinations of task ID values.

```

> ggplot(scores_lomo, aes(x = model_id, y = importance, fill = model_id)) +
  geom_col() +
  coord_flip() +

```

```
geom_hline(yintercept = 0, color = "black", linewidth = 0.25) +
facet_grid(cols = vars(target, horizon, location, target_end_date),
           scales = "free_x") +
labs(x = "Model ID", y = "Importance Score",
     title = "Model Importance by Task") +
scale_x_discrete(labels = function(x) gsub("[-_]", "-\\n", x)) +
theme(axis.text.x = element_text(angle = 90, hjust = 1, vjust = 0.5),
      panel.spacing.x = unit(0.5, "lines"), legend.position = "none")
```



We aggregate the importance scores for each model by averaging across all tasks. NA values are removed during the averaging process by setting the `na_action` argument to "drop".

```
> aggregate(scores_lomo, by = "model_id", na_action = "drop", fun = mean)
```

Overall model importance across tasks

```
-----
# A tibble: 3 x 2
  model_id      importance_score_mean
  <chr>          <dbl>
1 PSI-DICE          37.2
2 Flusight-baseline  28.4
3 MOBS-GLEAM_FLUH   -75
```

The results show that, overall, the model 'PSI-DICE' has the highest importance score, followed by 'Flusight-baseline' and 'MOBS-GLEAM_FLUH'. That is, 'PSI-DICE' contributes the most to improving the ensemble's predictive performance, whereas 'MOBS-GLEAM_FLUH', which has a negative score, detracts from the ensemble's performance. The low importance score of 'MOBS-GLEAM_FLUH' is mainly due to a substantially larger prediction error for Texas on the target end date of December 10, 2022, compared to other models, while its missing forecast for Massachusetts for November 26, 2022, was not factored into the evaluation. This

single large error significantly affected its contribution score.

Another approach to handling missing values is to use the "worst" option for `na_action`, which replaces missing values with the worst (i.e., minimum) score among the other models for the same task.

```
> aggregate(scores_lomo, by = "model_id", na_action = "worst", fun = mean)
```

```
Overall model importance across tasks
```

```
-----
# A tibble: 3 x 2
  model_id      importance_score_mean
  <chr>          <dbl>
1 Flusight-baseline      28.4
2 PSI-DICE             -17.6
3 MOBS-GLEAM_FLUH      -61.1
```

The results show that the importance score of 'Flusight-baseline' is unchanged because it has no missing forecasts. We observe that the importance score of 'PSI-DICE', which was previously positive, has now decreased to a negative value when compared to the evaluation using the "drop" option for `na_action`. Moreover, 'MOBS-GLEAM_FLUH' still ranks the lowest, but the importance score has increased. This change is related to the varying forecast accuracy across different tasks. For the target end date of November 26, 2022, in Massachusetts, most forecasts are relatively accurate. Thus, even if the 'MOBS-GLEAM_FLUH' is assigned the worst value of importance score for its missing forecast, including this value in the averaging is not detrimental to the overall importance metric; rather, it is more beneficial than excluding it. In contrast, for the target end date of December 10, 2022, in Texas, the forecasts have much larger errors across the board, and assigning the worst value of importance score to the missing forecast of 'PSI-DICE' in this task has a detrimental effect on averaging importance scores. This is because the scale of the importance scores is influenced by the magnitude of the prediction errors: for tasks with small errors, the scores remain moderate, while tasks with large errors can yield importance scores of much greater magnitude.

It is also possible to impute the missing scores with intermediate values by assigning the average importance scores of other models in the same task. This strategy may offer a more balanced trade-off by mitigating the influence of the missing data without overly penalizing or overlooking them.

```
> aggregate(scores_lomo, by = "model_id", na_action = "average", fun = mean)
```

```
Overall model importance across tasks
```

```
-----
# A tibble: 3 x 2
  model_id      importance_score_mean
  <chr>          <dbl>
1 Flusight-baseline      28.4
2 PSI-DICE             27.9
3 MOBS-GLEAM_FLUH      -56.2
```

6.3. Evaluation using LASOMO algorithm

We now demonstrate the use of the LASOMO algorithm for evaluating model importance. Since we explored the difference of `na_action` options in the previous LOMO example ([Section 6.2](#)), we focus on options for `subset_wt`, which specifies how weights are assigned to subsets of models when calculating importance scores, with `na_action` fixed to "drop".

The following code and corresponding outputs illustrate the evaluation using each weighting scheme.

```
> scores_lasomo_eq <- model_importance(
  forecast_data = forecast_data,
  oracle_output_data = target_data,
  ensemble_fun = "simple_ensemble",
  importance_algorithm = "lasomo",
  subset_wt = "equal"
)
> aggregate(scores_lasomo_eq, by = "model_id", na_action = "drop", fun = mean)
```

Overall model importance across tasks

```
-----
# A tibble: 3 x 2
  model_id      importance_score_mean
  <chr>          <dbl>
1 PSI-DICE          47.4
2 Flusight-baseline  24.3
3 MOBS-GLEAM_FLUH   -79.8
```

```
> scores_lasomo_perm <- model_importance(
  forecast_data = forecast_data,
  oracle_output_data = target_data,
  ensemble_fun = "simple_ensemble",
  importance_algorithm = "lasomo",
  subset_wt = "perm_based"
)
> aggregate(scores_lasomo_perm, by = "model_id", na_action = "drop", fun = mean)
```

Overall model importance across tasks

```
-----
# A tibble: 3 x 2
  model_id      importance_score_mean
  <chr>          <dbl>
1 PSI-DICE          44.8
2 Flusight-baseline  25.3
3 MOBS-GLEAM_FLUH   -78.6
```

In this example, there are only three models ($n = 3$), and the weights do not differ significantly between the two weighting schemes. Therefore, the resulting outputs show little difference. However, in general, with a larger number of models, the two weighting schemes may yield quite different importance scores for each model, as discussed in [Section 3.1](#).

In this section, we explored each component model's contribution to the ensemble accuracy

with only three models. An extensive application in more complex scenarios with larger ensembles is presented in our companion methodological paper (Kim *et al.* 2026).

It should be noted that the example presented here is designed for illustration purposes to demonstrate the use of the proposed software, and the results should not be interpreted as an authoritative evaluation of model performance. A comprehensive analysis in our prior methodological study (Kim *et al.* 2026) shows that many models, including the ‘MOBS-GLAM_FLUH’ model, are assessed as more important than the baseline model.

7. Computational complexity

This section describes the computational complexity of LOMO and LASOMO algorithms implemented in the package, depending on the numbers of ensemble component models and prediction tasks. We conducted a computational experiment using simulated data with a point forecast (‘median’). The execution time was measured for both algorithms across varying numbers of models and tasks. We also compared the execution times between sequential and multisession computing environments to demonstrate the efficiency of parallelization in handling computationally intensive tasks.

We performed all experiments on a machine running macOS 15 with an Apple M4 chip and 16 GB RAM under R version 4.4.3. Parallel computations were implemented using the **future** (multisession backend) with four worker processes. The execution times were recorded in seconds.

Figure 6 illustrates the computational runtime for four different numbers of prediction tasks (10, 20, 50, and 100) and varying numbers of models (ranging from 2 to 10) for both the LOMO and LASOMO algorithms under sequential and parallel execution modes. The scale of the y -axis approximately ranges from 0 to 12 seconds for LOMO and from 0 to 1110 seconds for LASOMO. Overall, the time needed to compute model importance scores increases with the number of models and tasks for both algorithms. Specifically, when the number of tasks is 10 or 20, the difference in computation time between sequential and parallel executions is not significant. However, as the number of prediction tasks rises to 50, the difference becomes more noticeable. The speedup from parallel execution relative to sequential execution is approximately 4-fold for LOMO in these settings of 10, 20, 50, and 100 prediction tasks, which is consistent with the use of four worker processes. Similarly, for LASOMO, a speedup of approximately 3-fold is observed across these settings, which is less than the 4-fold speedup observed with LOMO due to the heavier computational intensity of LASOMO. Nevertheless, the efficiency of parallel execution is more pronounced for LASOMO than for LOMO, particularly when eight or more models are involved in the ensemble construction and the number of prediction tasks is large.

Theoretically, the computational complexity of LOMO and LASOMO algorithms is $O(t \cdot n)$ and $O(t \cdot 2^n)$, respectively, where n is the number of models and t is the number of prediction tasks. However, the observations from our computational experiment in the sequential execution mode exhibited slight discrepancies from these theoretical complexities, under a limited setting involving between 2 and 10 models and 10, 20, 50, and 100 prediction tasks. While the observed runtimes showed linear growth with respect to the number of tasks for both algorithms, consistent with the theoretical complexities, the growth with respect to the number

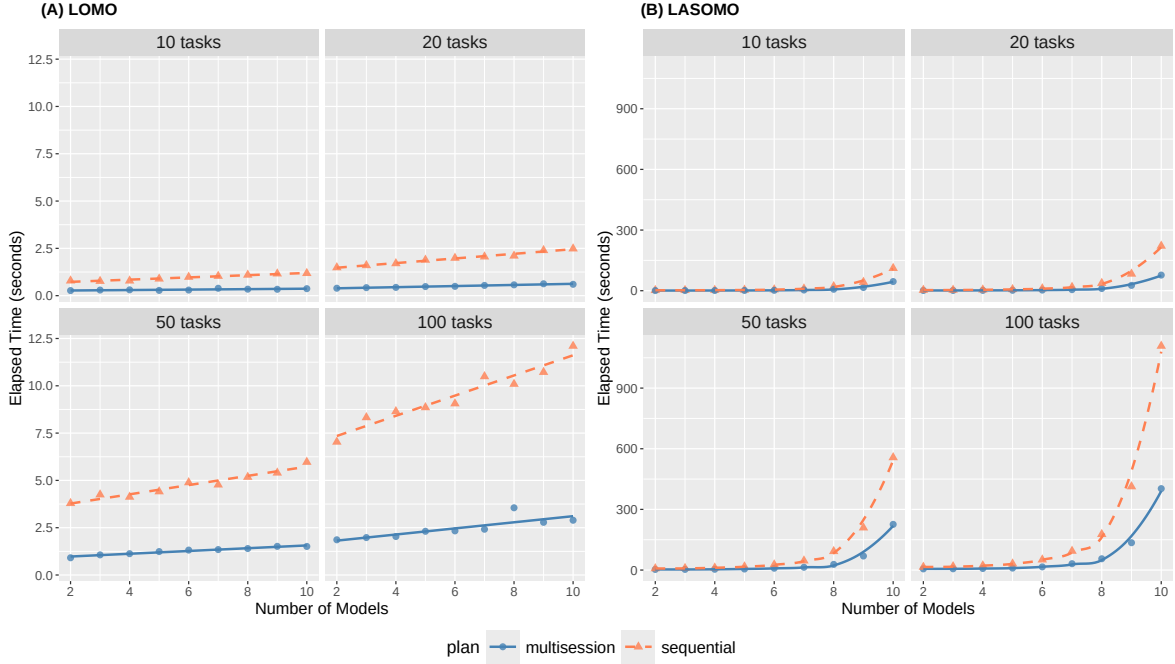
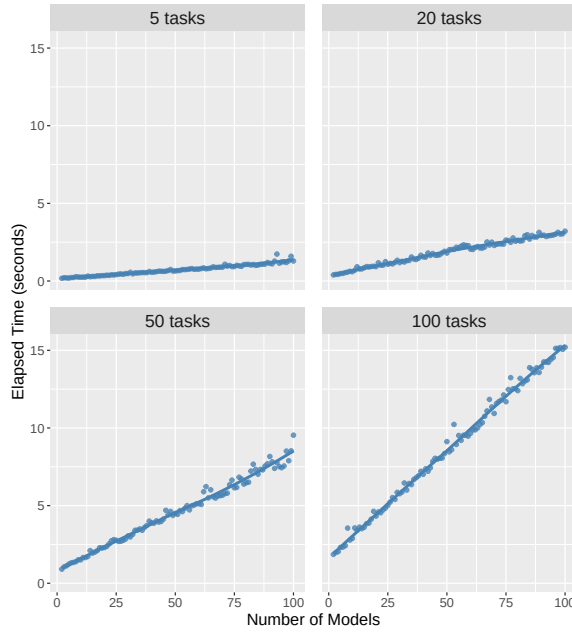


Figure 6: Runtime to compute model importance scores for point predictions by number of ensemble component models (ranging from 2 to 10) and number of prediction tasks (10, 20, 50, and 100) under sequential and parallel execution modes (multisession backend, with four workers) for LOMO and LASOMO algorithms. Each point represents the elapsed time (in seconds) for a given number of models, while lines (solid/dashed) show the overall trend fitted to the observations. The scale of the y -axis is different between the two algorithms.

of models is sublinear for LOMO and slower than the expected exponential for LASOMO. This deviation from the theoretical patterns is likely due to the practical efficiency of the implementations, including shared computation. Specifically, once the ensemble predictions are computed, the importance scores are derived from these predictions without needing to recompute the ensemble for each model. It should be noted that the scaling increment and ratio between adjacent values of n in LOMO and LASOMO, respectively, is gradually increasing, which implies the empirical complexity is approaching the theoretical complexity with the number of models. However, the execution time scales much more rapidly with the number of models than the theoretical complexities, notably in LASOMO, as larger sub-ensembles require more expensive evaluation and scoring operations across all prediction tasks as well as larger intermediate objects. That is, the theoretical complexity does not factor in subset size, which does, in fact, impact the computational speed in practice. This phenomenon is also observed even in the parallel execution setting (Figure 7b).

We further explored computational feasibility, focusing on the parallel execution mode (Figure 7). We observe that, in LOMO, the elapsed time increases linearly with the number of models and the rate of increase depends on the number of tasks, with a steeper increase as the number of tasks grows. In contrast, in LASOMO, the elapsed time increases much more rapidly as the ensemble size grows. In particular, when the number of models exceeds 12, it takes more than 20 minutes to compute the importance scores, even with fixed 5 prediction tasks. It

(a) LOMO



(b) LASOMO

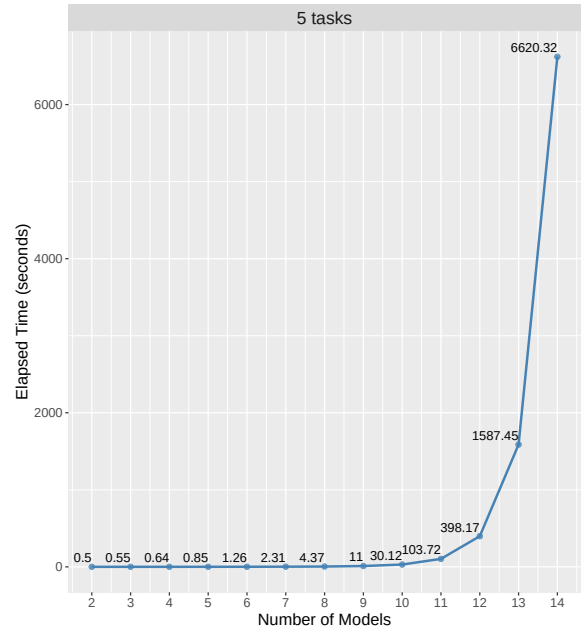


Figure 7: Runtime to compute model importance scores for point predictions by number of ensemble component models and prediction tasks during parallel execution via multisession backend. Each dot represents the elapsed time (in seconds) for a given number of models. For LOMO, the number of models evaluated varies from 2 to 100 for each 5, 20, 50, and 100 prediction tasks. The fitted lines illustrate the trend of the observations. For LASOMO, the number of models evaluated varies from 2 to 14 while fixing 5 prediction tasks due to exponentially growing computational intensity with the number of models. The exact elapsed time is shown for LASOMO, which captures the steep increase in runtime as the number of models grows, even before the high jump occurs with models over 12.

is because the number of subsets of models that need to be evaluated in LASOMO grows exponentially with the number of models, leading to a rapidly rising computational intensity. This observation highlights the trade-off between the comprehensiveness of the LASOMO algorithm and its computational challenges in practice, particularly with a large number of models.

8. Implementation and availability

The **modelimportance** package is implemented in R and distributed via CRAN and GitHub, under the MIT license (<https://github.com/mkim425/modelimportance>). We conducted unit tests using the **testthat** package (Wickham 2011) to verify that all inputs and outputs are properly formatted and ensure that all functions work correctly as expected, including those used internally. We also performed continuous integration testing using GitHub Actions to maintain functionality across platforms, including Windows, macOS, and Linux. Integrated GitHub Action, we employed the **lintr** package (Hester *et al.* 2025) to maintain code quality

and detect potential issues, and the Air formatter (Posit Software, PBC 2025) to ensure consistent code style across the codebase. All code changes were systematically reviewed by fellow team members, and this enhanced reliability.

9. Summary and discussion

Multi-model ensemble forecasts often provide better accuracy and robustness than single models, and are widely used in decision-making and policy planning across various domains. The contribution of each component model to the accuracy of the ensemble depends on its own unique characteristics. The **modelimportance** package enables the quantification of the value that each component model adds to the ensemble performance in different evaluation contexts.

In the example analysis, we showed the package workflow using a simple example dataset specifically designed solely to demonstrate our package’s capabilities; thus the results should not be taken as a definitive assessment of the models. A more complete model importance analysis is provided in our companion methodological paper (Kim *et al.* 2026).

The primary `model_importance()` function returns an S3 object of class `model_imp_tbl` of tabular data containing component models and their importance metrics. Users are given a choice of ensemble methods, model importance algorithms (either LOMO or LASOMO), and options to handle missing values. These features enable the package to serve as a versatile tool to aid collaborative efforts to construct an effective ensemble model across a wide range of forecasting tasks. We note that unit testing with continuous integration ensures the reliability of all functions and the overall quality of code across multiple platforms.

The **modelimportance** package still has several areas in which it may be enhanced. Namely, although the package currently supports four different output types (‘mean’, ‘median’, ‘quantile’, and ‘pmf’), other output types are widely used in practice. For example, the ‘sample’ output type is commonly used in the US Flu Scenario Modeling Hub (Flu Scenario Modeling Hub 2024). This format includes multiple simulated values (samples) from the forecast distribution. Support for the ‘sample’ output type is under consideration for future releases, and, in general, extensions to support more output types would aim to broaden the scope of applications in real-world forecasting tasks.

Acknowledgements

We acknowledge Zhian N. Kamvar for debugging and resolving coding issues while developing the package. We are also grateful to Matthew Cornell for his advice on unit testing, which greatly helped us improve the structure and testing our code with a solid understanding of unit testing practices. Moreover, we would like to thank the hubverse development team for their data standards, on which our package is based.

Appendix

A. Weights for subsets in LASOMO

In the LASOMO algorithm, two weighting schemes are available for subsets of models in the calculation of model importance scores: equal weights and permutation-based weights.

Let n be the total number of models and k be the size of a subset that does not include the model being evaluated. The formulas for the weights under each scheme are as follows:

$$w^{\text{eq}} = \frac{1}{2^{n-1} - 1},$$

$$w^{\text{perm}} = \frac{1}{(n-1) \binom{n-1}{k}},$$

where the superscripts “eq” and “perm” denote the equal and permutation-based weighting schemes, respectively. The maximum weight under the permutation-based scheme occurs when $k = n - 1$, which is $1/(n - 1)$. The minimum weight occurs when the subset size is around $(n - 1)/2$ (i.e., $k = \lfloor (n - 1)/2 \rfloor$), which is approximately $\frac{\sqrt{\pi(n-1)/2}}{(n-1)2^{n-1}}$ by Stirling’s approximation.

Given a fixed mid-sized subset, as n increases, the weight assigned to this subset under the equal weighting scheme decreases at a rate of $O(1/2^n)$, while under the permutation-based scheme, it decreases at a much faster rate of $O(1/(\sqrt{n} 2^n))$. This indicates that as the number of models grows, that mid-sized subset becomes significantly less influential in determining model importance scores when using the permutation-based weighting scheme compared to the equal weighting scheme.

On the other hand, for subsets of extreme sizes (e.g., $k = 1$ or $k = n - 1$), the weights under permutation-based weighting scheme decrease only at $O(1/n)$, much slower under the equal weighting scheme. This implies that in scenarios with a large number of models, the contributions of these extreme-sized subsets play a relatively larger role in the calculation of model importance scores when using permutation-based weights compared to the equal weighting approach.

References

- Bengtsson H (2021). “A Unifying Framework for Parallel and Distributed Processing in R using Futures.” *The R Journal*, **13**(2), 208–227. doi:10.32614/RJ-2021-048. URL <https://doi.org/10.32614/RJ-2021-048>.
- Bosse NI, Gruson H, Cori A, van Leeuwen E, Funk S, Abbott S (2024). “Evaluating forecasts with scoringutils in R.” *arXiv preprint arXiv:2205.07090v2*.
- Bracher J, Ray EL, Gneiting T, Reich NG (2021). “Evaluating epidemic forecasts in an interval format.” *PLOS Computational Biology*, **17**(2), e1008618. doi:10.1371/journal.pcbi.1008618.
- Brooks LC, Farrow DC, Hyun S, Tibshirani RJ, Rosenfeld R (2018). “Nonmechanistic forecasts of seasonal influenza with iterative one-week-ahead distributions.” *PLOS Computational Biology*, **14**(6), e1006134.

- Chen T, He T, Benesty M, Khotilovich V, Tang Y, Cho H, Chen K, Mitchell R, Cano I, Zhou T, Li M, Xie J, Lin M, Geng Y, Li Y, Yuan J (2024). **xgboost**: *Extreme Gradient Boosting*. R package version 1.7.7.1, URL <https://CRAN.R-project.org/package=xgboost>.
- Consortium of Infectious Disease Modeling Hubs (2024). “The hubverse: open tools for collaborative forecasting.” Accessed: 2025-01-14, URL <https://hubverse.io>.
- Covert I, Lundberg SM, Lee SI (2020). “Understanding global feature contributions with additive importance measures.” *Advances in neural information processing systems*, **33**, 17212–17223.
- Cramer EY, Huang Y, Wang Y, Ray EL, Cornell M, Bracher J, Brennen A, Rivadeneira AJC, Gerding A, House K, *et al.* (2022). “The United States COVID-19 forecast hub dataset.” *Scientific data*, **9**(1), 462.
- Flu Scenario Modeling Hub (2024). “Flu Scenario Modeling Hub.” Accessed: 2025-07-25, URL <https://fluscenariomodelinghub.org/index.html>.
- Gneiting T, Raftery AE (2005). “Weather forecasting with ensemble methods.” *Science*, **310**(5746), 248–249.
- Gneiting T, Raftery AE (2007). “Strictly Proper Scoring Rules, Prediction, and Estimation.” *Journal of the American Statistical Association*, **102**(477), 359–378. doi: [10.1198/016214506000001437](https://doi.org/10.1198/016214506000001437).
- Hamner B, Frasco M (2018). **Metrics**: *Evaluation Metrics for Machine Learning*. R package version 0.1.4, URL <https://CRAN.R-project.org/package=Metrics>.
- Hastie T, Tibshirani R, Friedman J (2001). *The Elements of Statistical Learning*. Springer Series in Statistics. Springer New York Inc., New York, NY, USA.
- He K, Yang Q, Ji L, Pan J, Zou Y (2023). “Financial Time Series Forecasting with the Deep Learning Ensemble Model.” *Mathematics*, **11**(4). doi: [10.3390/math11041054](https://doi.org/10.3390/math11041054).
- Hester J, Angly F, Hyde R, Chirico M, Ren K, Rosenstock A, Patil I (2025). **lintr**: *A ‘Linter’ for R Code*. R package version 3.2.0, URL <https://CRAN.R-project.org/package=lintr>.
- Jordan A, Krüger F, Lerch S (2019). “Evaluating Probabilistic Forecasts with **scoringRules**.” *Journal of Statistical Software*, **90**(12), 1–37. doi: [10.18637/jss.v090.i12](https://doi.org/10.18637/jss.v090.i12).
- Jordan A Guerra, Sophie A Murray, D Shaun Bloomfield, Peter T Gallagher (2020). “Ensemble forecasting of major solar flares: methods for combining models.” *J. Space Weather Space Clim.*, **10**, 38. doi: [10.1051/swsc/2020042](https://doi.org/10.1051/swsc/2020042).
- Kerr M, Borchering R, Rivadeneira AC, Contamin L, Funk S, Hochheiser H, Howerton E, Krystalli A, Shandross L, Reich NG, *et al.* (2025). “Coordinating collaborative infectious disease modeling projects with the hubverse.” *medRxiv*.
- Kim M, Ray EL, Reich NG (2026). “Beyond forecast leaderboards: Measuring individual model importance based on contribution to ensemble accuracy.” *International Journal of Forecasting*. doi: <https://doi.org/10.1016/j.ijforecast.2025.12.006>.

- Krystalli A, Shandross L (2025). **hubUtils**: Core 'hubverse' Utilities. R package version 0.6.0.9000, commit 71964f7cd6f79e48f3038952fed8f742a7fe6a5c, URL <https://github.com/hubverse-org/hubUtils>.
- Kuhn M (2008). "Building Predictive Models in R Using the **caret** Package." *Journal of Statistical Software*, **28**(5), 1–26. doi:10.18637/jss.v028.i05. URL <https://www.jstatsoft.org/index.php/jss/article/view/v028i05>.
- Liaw A, Wiener M (2002). "Classification and Regression by **randomForest**." *R News*, **2**(3), 18–22. URL <https://CRAN.R-project.org/doc/Rnews/>.
- Lipiecki A, Uniejewski B, Weron R (2024). "Postprocessing of point predictions for probabilistic forecasting of day-ahead electricity prices: The benefits of using isotonic distributional regression." *Energy Economics*, **139**, 107934.
- Lipiecki A, Weron R (2025). "**PostForecasts.jl**: A Julia package for probabilistic forecasting by postprocessing point predictions." *SoftwareX*, **31**, 102200.
- Lundberg SM, Erion G, Chen H, DeGrave A, Prutkin JM, Nair B, Katz R, Himmelfarb J, Bansal N, Lee SI (2020). "From local explanations to global understanding with explainable AI for trees." *Nature machine intelligence*, **2**(1), 56–67.
- Lundberg SM, Lee SI (2017). "A unified approach to interpreting model predictions." *Advances in neural information processing systems*, **30**.
- Lutz CS, Huynh MP, Schroeder M, Anyatonwu S, Dahlgren FS, Danyluk G, Fernandez D, Greene SK, Kipshidze N, Liu L, Mgbere O, McHugh LA, Myers JF, Siniscalchi A, Sullivan AD, West N, Johansson MA, Biggerstaff M (2019). "Applying infectious disease forecasting to public health: a path forward using influenza forecasting examples." *BMC Public Health*, **19**(1), 1659. doi:10.1186/s12889-019-7966-8.
- O'Hara-Wild M, Hyndman R, Wang E (2024). **fable**: Forecasting Models for Tidy Time Series. R package version 0.4.0, <https://github.com/tidyverts/fable>, URL <https://fable.tidyverts.org>.
- Posit Software, PBC (2025). "**Air**: An R formatter and language server." Version 0.8.1, URL <https://github.com/posit-dev/air>.
- Ranjan R, Gneiting T (2010). "Combining probability forecasts." *Journal of the Royal Statistical Society Series B: Statistical Methodology*, **72**(1), 71–91.
- Ray EL, Reich NG (2018). "Prediction of infectious disease epidemics via weighted density ensembles." *PLOS Computational Biology*, **14**(2), e1005910. doi:10.1371/journal.pcbi.1005910.
- Ray EL, Sweger B, Contamin L (2025). **hubExamples**: Example Hub Data. R package version 0.1.0.9000, commit df75591bff3d68c3ecb1cad4798b1334230203f7, URL <https://github.com/hubverse-org/hubExamples>.
- Reich N, Ray E, Bosse N, Cornell M, Sweger B, Roosa K (2025). **hubEvals**: Basic tools for scoring hubverse forecasts. R package version 0.0.0.9001, commit 0329cb4f94f833c495bc3001cbac82b103f35d2, URL <https://github.com/hubverse-org/hubEvals>.

- Reich NG, McGowan CJ, Yamana TK, Tushar A, Ray EL, Osthus D, Kandula S, Brooks LC, Crawford-Crudell W, Gibson GC, Moore E, Silva R, Biggerstaff M, Johansson MA, Rosenfeld R, Shaman J (2019). “Accuracy of real-time multi-model ensemble forecasts for seasonal influenza in the U.S.” *PLOS Computational Biology*, **15**(11), e1007486. doi: [10.1371/journal.pcbi.1007486](https://doi.org/10.1371/journal.pcbi.1007486).
- Ridgeway G, Developers G (2024). **gbm**: *Generalized Boosted Regression Models*. R package version 2.2.2, URL <https://CRAN.R-project.org/package=gbm>.
- Shandross L, Howerton E, Contamin L, Hochheiser H, Krystalli A, of Infectious Disease Modeling Hubs C, Reich NG, Ray EL (2026a). “Multi-Model Ensembles in Infectious Disease and Public Health: Methods, Interpretation, and Implementation in R.” *Statistics in Medicine*, **45**(1-2), e70333.
- Shandross L, Howerton E, Ray EL (2026b). **hubEnsembles**: *Ensemble Methods for Combining Hub Model Outputs*. R package version 1.0.0.9000, commit 29f53ff7e983321b753c61fdf4d355dac9ba6fcf, URL <https://github.com/hubverse-org/hubEnsembles>.
- Shapley LS (1953). “A value for n-person games.” *Contribution to the Theory of Games*, **2**.
- Sun S, Wang S, Wei Y (2020). “A new ensemble deep learning approach for exchange rates forecasting and trading.” *Advanced Engineering Informatics*, **46**, 101160. doi: <https://doi.org/10.1016/j.aei.2020.101160>.
- Vaughan D, Dancho M (2022). **furrr**: *Apply Mapping Functions in Parallel using Futures*. R package version 0.3.1, URL <https://CRAN.R-project.org/package=furrr>.
- Viboud C, Sun K, Gaffey R, Ajelli M, Fumanelli L, Merler S, Zhang Q, Chowell G, Simonsen L, Vespignani A (2018). “The RAPIDD ebola forecasting challenge: Synthesis and lessons learnt.” *Epidemics*, **22**, 13–21. doi: [10.1016/j.epidem.2017.08.002](https://doi.org/10.1016/j.epidem.2017.08.002).
- Wang X, Hyndman RJ, Li F, Kang Y (2023). “Forecast combinations: An over 50-year review.” *International Journal of Forecasting*, **39**(4), 1518–1547.
- Wickham H (2011). “**testthat**: Get Started with Testing.” *The R Journal*, **3**, 5–10. URL https://journal.r-project.org/archive/2011-1/RJournal_2011-1_Wickham.pdf.
- Wickham H (2016). **ggplot2**: *Elegant Graphics for Data Analysis*. Springer-Verlag New York. ISBN 978-3-319-24277-4. URL <https://ggplot2.tidyverse.org>.
- Xie Y (2025). **knitr**: *A General-Purpose Package for Dynamic Report Generation in R*. R package version 1.51, URL <https://yihui.org/knitr/>.
- Yan Y (2024). **MLmetrics**: *Machine Learning Evaluation Metrics*. R package version 1.1.3, URL <https://CRAN.R-project.org/package=MLmetrics>.
- Zhu H (2024). **kableExtra**: *Construct Complex Table with ‘kable’ and Pipe Syntax*. R package version 1.4.0, URL <https://CRAN.R-project.org/package=kableExtra>.

Affiliation:

Minsu Kim

E-mail: minsukim@umass.edu

Li Shandross

Evan L. Ray

Nicholas G. Reich